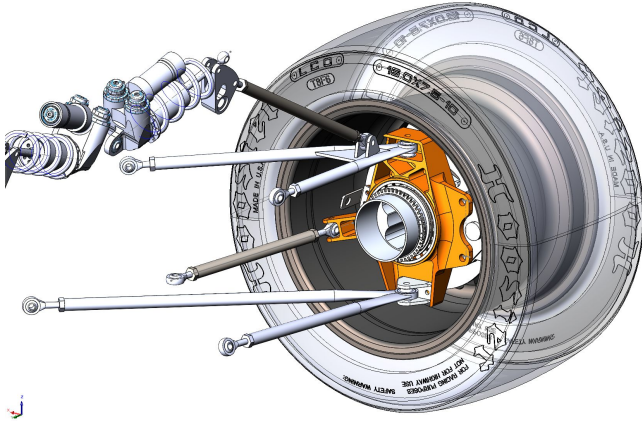


Upright ("Steering Knuckle") Project



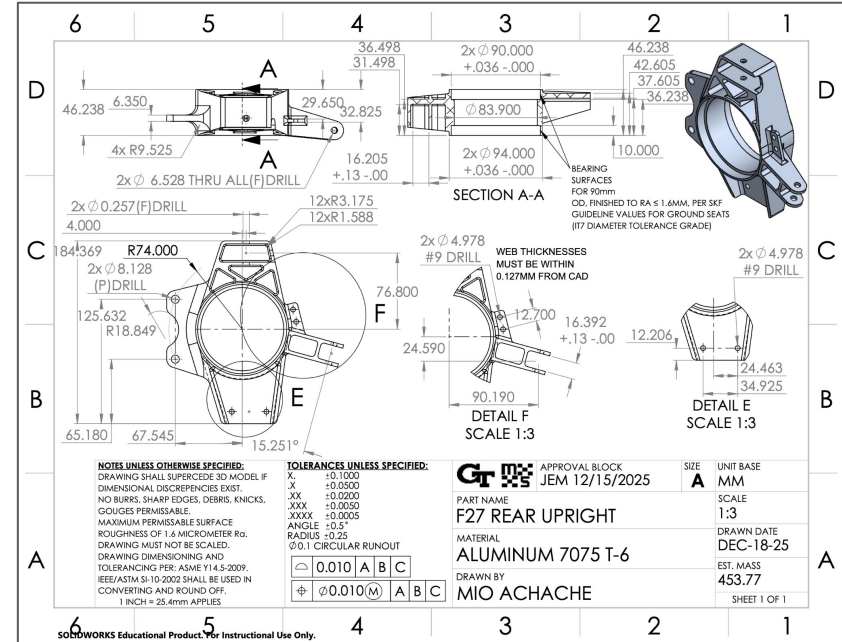
Full Suspension Assembly in CAD with upright



Final Manufactured Part

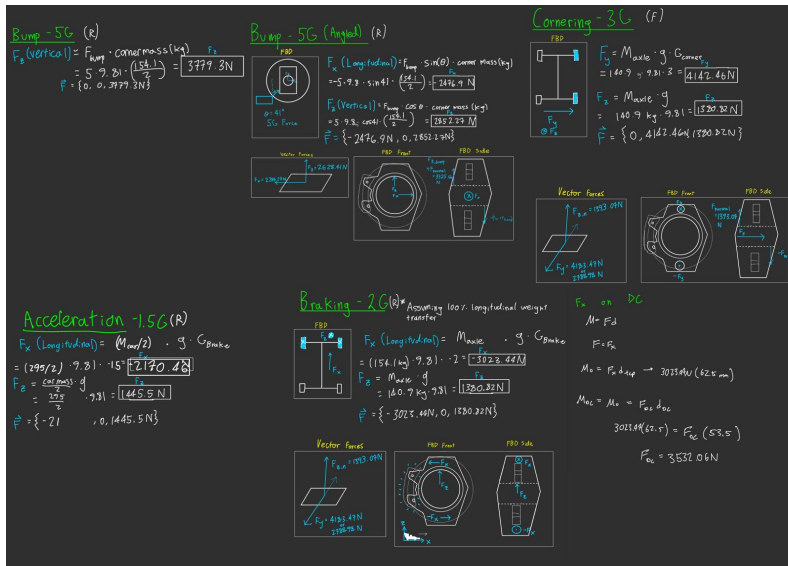
Summary

- Designed and optimized rear suspension uprights for GT Motorsports' 2027 Formula SAE car using SolidWorks, specifying aluminum 7075-T6 with GD&T tolerancing per ASME Y14.5-2009
- Ran Ansys FEA across five load cases (5G bump, 5G angled bump, 3G cornering, 2G braking, 1.5G acceleration) with load values derived from vehicle IMU data via custom MATLAB scripts
- Maintained minimum 1.5 safety factor across all conditions and coordinated CNC manufacturing on Haas mill to produce final parts



Final Part Drawing

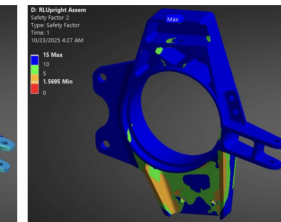
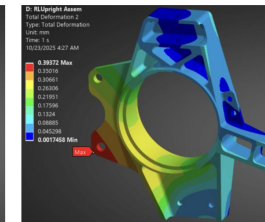
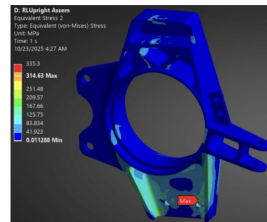
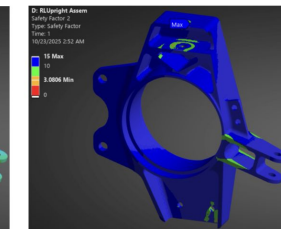
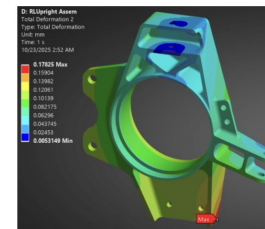
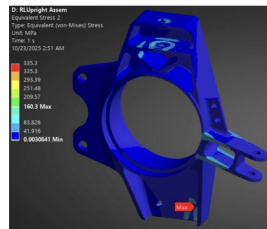
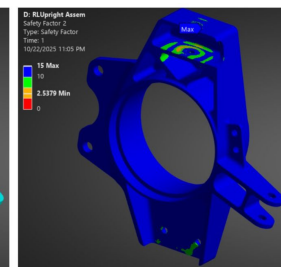
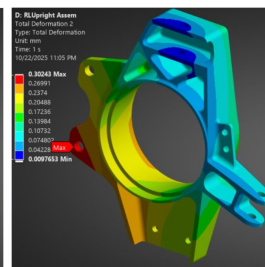
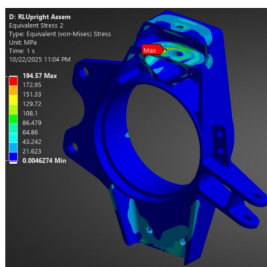
Upright Project Continued



Load Case Derivations Methodology

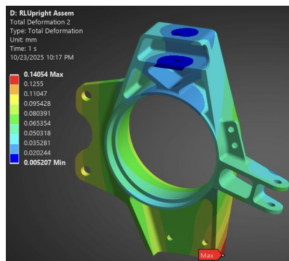
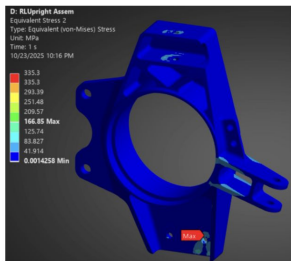
Load Case	Max Stress	Max Deformation	Min Safety Factor	Critical Failure Mode (Y/N)
5G Vertical Bump	194.57 MPa	0.30243 mm	2.5379	Yes
5G Angled Bump	160.3 MPa	0.17825 mm	3.0806	Yes
3G Corner	314.63 MPa	0.39372 mm	1.5695	Yes
2G Braking	254.11 MPa	0.53383 mm	1.9433	Yes
1.5G Acceleration	166.85 MPa	0.14054 mm	2.9596	Yes

FEA Results

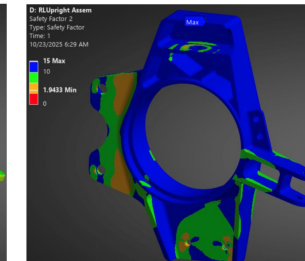
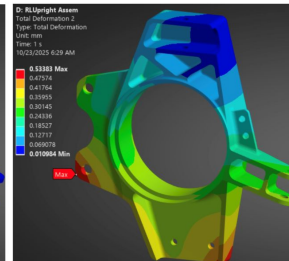
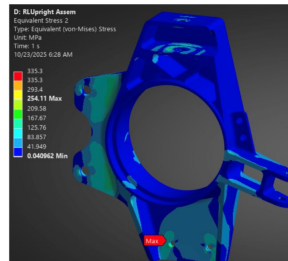
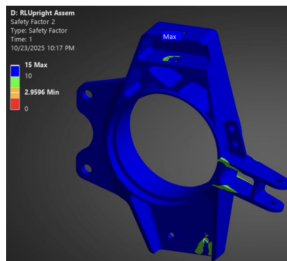


3G Cornering

Upright Project Continued

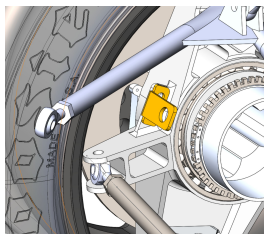


1.5G Acceleration



2G Braking

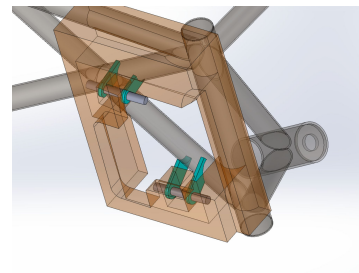
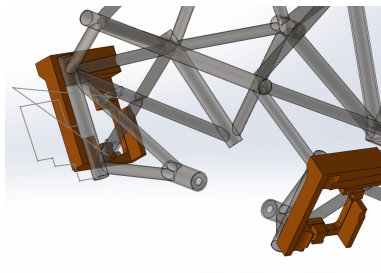
Miscellaneous Side Projects



Wheel Speed Sensor Mounts

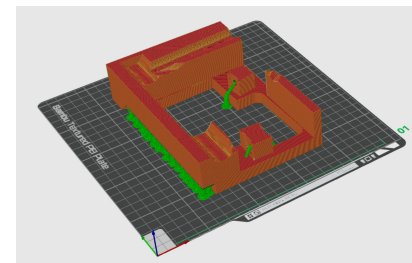


Solidworks sheet metal, waterjetted, then bent with sheet metal brake



Chassis Tab Alignment Jig

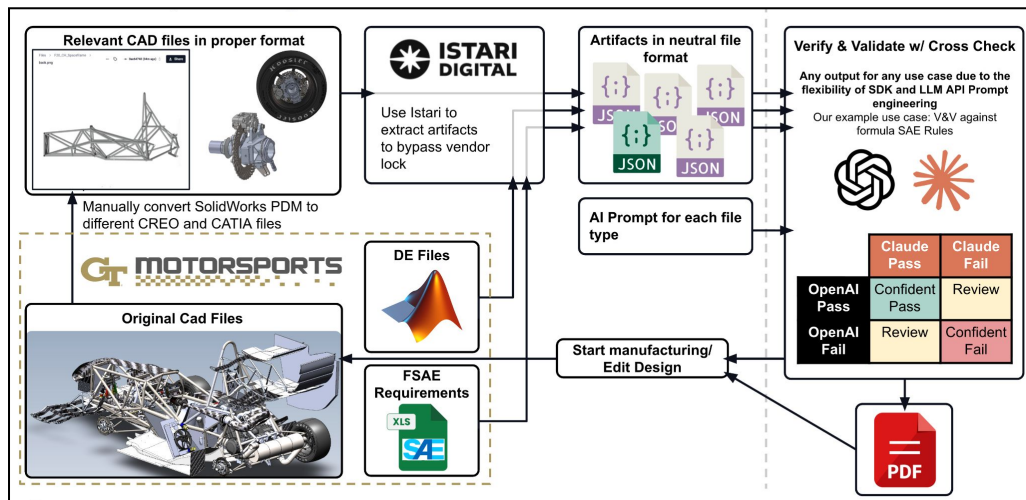
Designed and 3d Printed to aid Chassis team in welding remaining toe and pushrod tabs



Automated Verification & Validation Project

Summary

- Built automated Python pipelines using Istari's SDK to centralize CAD (CATIA, Creo) and model data (MATLAB, Nastran) into a proprietary V&V tool for vehicle performance analysis and airworthiness auditing
- Designed "dual LLM verification" system using Claude and GPT to cross-check validation results, cutting LLM hallucinations by ~75% and reducing manual V&V process time by ~75%
- Led engineering file generation for federal customer success project, distributing files to prime contractors including RTX, MIT Lincoln Lab, and MITRE



Project Workflow



ID	Requirement Name	Unit	Min	Max	How Verified
Req-1	Brake Rotor Surface Roughness	in	0.000126	0.0003937	CAD material property
Req-2	Brake Rotor Mass	lb	0.8	1	CAD material property
Req-3	Brake Rotor Maximum Temperature	F	392	572	CAD material property
Req-4	Brake Rotor Inner Diameter	in	4.7	2.5	CAD geometry
Req-5	Brake Rotor Outer Diameter	in	8.5	10	CAD geometry
Req-6	Front Hub Inner Diameter	in	1.2	1.5	CAD geometry
Req-7	Front Hub Outer Diameter	in	2	2.35	CAD geometry
Req-8	Front Hub Length	in	2.5	2.5	CAD geometry
Req-9	Wheel Speed Sensing Ring Groove Count	ignore	40	60	CAD pattern count
Req-10	Wheel Speed Sensing Inner diameter	in	1.25	1.5	CAD geometry
Req-11	Upright Inner Borehole Diameter	in	1.38	4	CAD geometry
Req-12	Upright Material	ignore			CAD geometry
Req-13	Tire Outer Diameter	in	16	16	CAD geometry
Req-14	Tire Inner Diameter	in	19	10	CAD geometry
Req-15	Tire Volume	in^3			CAD measured dimension
Req-16	Tire Maximum PSI	psi	8	16	From MATLAB Script
Req-17	Max Lateral Force	newtons	1790	2100	From MATLAB Script
Req-18	Wheel Nut Inner Diameter	in	0.75	1	CAD geometry
Req-19	Rim Outer Diameter	in	10	10	CAD geometry
Req-20	Rim Max RPM	rpm	10000	infinity	CAD geometry
Req-21	Sensor Bracket Sensor Mount Diameter	in	0.5	0.5	CAD geometry
Req-22	Sensor Bracket Mounting Bolt Diameter	in	0.25	0.25	CAD geometry
Req-23	Sensor Diameter	in	0.5	0.6	CAD geometry
Req-24	Sensor Height	in	2	2.5	CAD geometry
Req-25	Intake Restrictor Diameter (Gas Fueled)	in	0.7	0.9	CAD geometry
Req-26	Intake Restrictor Diameter (E85 Fueled)	in	0.6	0.8	CAD geometry
Req-27	Front Wing Max Vert Deflection Under 100kg Load	in	0	0.4	CAD geometry
Req-28	Front Wing Mounting Tabs Minimum Thickness	in	0.063	infinity	CAD geometry
Req-29	Chassis Minimum Ride Height	in	2.95	infinity	CAD geometry

PART NAME	PARAMETER	VALUE	REQUIREMENT	MIN	MAX	STATUS	REASON
OZ_wheel	Rim Outer Diameter	10 in	Rim Outer Diameter	10 in	10 in	PASS	Rim outer diameter of 10 inches meets the exact

The figure displays a 3D model of a complex truss structure, likely a bridge or a large industrial component, rendered in a light gray color. The structure features a long, low-profile main span supported by a central vertical column and two side columns, with various diagonal bracing members. Below the model is a table summarizing the test results.

Test Name	Parameter	Value	Requirement	Min	Max	Status	Reason
FOR_Or_Space Frame	Check Maximum Side Height	9 in	Check Maximum Side Height	2.85 in	10 in	PASS	Parameter Passed in GSEA parameters. J was within 10 in to 10 in and within 10 in within 10 in checked bounds.

Program Example Outputs

The screenshot displays a JupyterLab environment with a file named `03_validate.py` open in the editor. The script is a Python program that uses `argparse` to handle command-line arguments. It defines a `validation_mode` argument with a default value of `'help'` and a `choices` parameter set to `['1', '2', '3']`. The script includes a `validation_message` function that checks if the input is a valid choice and prints a message accordingly. The `main` function calls `validation_message` with the `validation_mode` argument. The terminal output shows the script being executed with the argument `1`, resulting in a success message: `Validation successful. Choice is valid.`

```

1 #!/usr/bin/env python3
2
3 import argparse
4
5 def validation_message(validation_mode):
6     if validation_mode in ['1', '2', '3']:
7         print(f"Validation successful. Choice is valid.")
8     else:
9         print(f"Validation failed. Choice is not valid.")
10
11 def main():
12     parser = argparse.ArgumentParser()
13     parser.add_argument(
14         "-m", "--validation-mode",
15         type=str,
16         default="help",
17         choices=['1', '2', '3'],
18         help="Validation mode (1, 2, or 3).",
19     )
20     args = parser.parse_args()
21     validation_message(args.validation_mode)
22
23 if __name__ == "__main__":
24     main()
25
26 # Run the script with the argument 1
27 python3 03_validate.py -m 1
28
29 # Output:
30 Validation successful. Choice is valid.
31
32 # Run the script with the argument 2
33 python3 03_validate.py -m 2
34
35 # Output:
36 Validation successful. Choice is valid.
37
38 # Run the script with the argument 3
39 python3 03_validate.py -m 3
40
41 # Output:
42 Validation successful. Choice is valid.
43
44 # Run the script with the argument help
45 python3 03_validate.py -m help
46
47 # Output:
48 Validation failed. Choice is not valid.
49
50 # Run the script with the argument invalid
51 python3 03_validate.py -m invalid
52
53 # Output:
54 Validation failed. Choice is not valid.
55
56 # Run the script with the argument 1
57 python3 03_validate.py -m 1
58
59 # Output:
60 Validation successful. Choice is valid.
61
62 # Run the script with the argument 2
63 python3 03_validate.py -m 2
64
65 # Output:
66 Validation successful. Choice is valid.
67
68 # Run the script with the argument 3
69 python3 03_validate.py -m 3
70
71 # Output:
72 Validation successful. Choice is valid.
73
74 # Run the script with the argument help
75 python3 03_validate.py -m help
76
77 # Output:
78 Validation failed. Choice is not valid.
79
80 # Run the script with the argument invalid
81 python3 03_validate.py -m invalid
82
83 # Output:
84 Validation failed. Choice is not valid.
85
86 # Run the script with the argument 1
87 python3 03_validate.py -m 1
88
89 # Output:
90 Validation successful. Choice is valid.
91
92 # Run the script with the argument 2
93 python3 03_validate.py -m 2
94
95 # Output:
96 Validation successful. Choice is valid.
97
98 # Run the script with the argument 3
99 python3 03_validate.py -m 3
100
101 # Output:
102 Validation successful. Choice is valid.
103
104 # Run the script with the argument help
105 python3 03_validate.py -m help
106
107 # Output:
108 Validation failed. Choice is not valid.
109
110 # Run the script with the argument invalid
111 python3 03_validate.py -m invalid
112
113 # Output:
114 Validation failed. Choice is not valid.
115
116 # Run the script with the argument 1
117 python3 03_validate.py -m 1
118
119 # Output:
120 Validation successful. Choice is valid.
121
122 # Run the script with the argument 2
123 python3 03_validate.py -m 2
124
125 # Output:
126 Validation successful. Choice is valid.
127
128 # Run the script with the argument 3
129 python3 03_validate.py -m 3
130
131 # Output:
132 Validation successful. Choice is valid.
133
134 # Run the script with the argument help
135 python3 03_validate.py -m help
136
137 # Output:
138 Validation failed. Choice is not valid.
139
140 # Run the script with the argument invalid
141 python3 03_validate.py -m invalid
142
143 # Output:
144 Validation failed. Choice is not valid.
145
146 # Run the script with the argument 1
147 python3 03_validate.py -m 1
148
149 # Output:
150 Validation successful. Choice is valid.
151
152 # Run the script with the argument 2
153 python3 03_validate.py -m 2
154
155 # Output:
156 Validation successful. Choice is valid.
157
158 # Run the script with the argument 3
159 python3 03_validate.py -m 3
160
161 # Output:
162 Validation successful. Choice is valid.
163
164 # Run the script with the argument help
165 python3 03_validate.py -m help
166
167 # Output:
168 Validation failed. Choice is not valid.
169
170 # Run the script with the argument invalid
171 python3 03_validate.py -m invalid
172
173 # Output:
174 Validation failed. Choice is not valid.
175
176 # Run the script with the argument 1
177 python3 03_validate.py -m 1
178
179 # Output:
180 Validation successful. Choice is valid.
181
182 # Run the script with the argument 2
183 python3 03_validate.py -m 2
184
185 # Output:
186 Validation successful. Choice is valid.
187
188 # Run the script with the argument 3
189 python3 03_validate.py -m 3
190
191 # Output:
192 Validation successful. Choice is valid.
193
194 # Run the script with the argument help
195 python3 03_validate.py -m help
196
197 # Output:
198 Validation failed. Choice is not valid.
199
200 # Run the script with the argument invalid
201 python3 03_validate.py -m invalid
202
203 # Output:
204 Validation failed. Choice is not valid.
205
206 # Run the script with the argument 1
207 python3 03_validate.py -m 1
208
209 # Output:
210 Validation successful. Choice is valid.
211
212 # Run the script with the argument 2
213 python3 03_validate.py -m 2
214
215 # Output:
216 Validation successful. Choice is valid.
217
218 # Run the script with the argument 3
219 python3 03_validate.py -m 3
220
221 # Output:
222 Validation successful. Choice is valid.
223
224 # Run the script with the argument help
225 python3 03_validate.py -m help
226
227 # Output:
228 Validation failed. Choice is not valid.
229
230 # Run the script with the argument invalid
231 python3 03_validate.py -m invalid
232
233 # Output:
234 Validation failed. Choice is not valid.
235
236 # Run the script with the argument 1
237 python3 03_validate.py -m 1
238
239 # Output:
240 Validation successful. Choice is valid.
241
242 # Run the script with the argument 2
243 python3 03_validate.py -m 2
244
245 # Output:
246 Validation successful. Choice is valid.
247
248 # Run the script with the argument 3
249 python3 03_validate.py -m 3
250
251 # Output:
252 Validation successful. Choice is valid.
253
254 # Run the script with the argument help
255 python3 03_validate.py -m help
256
257 # Output:
258 Validation failed. Choice is not valid.
259
260 # Run the script with the argument invalid
261 python3 03_validate.py -m invalid
262
263 # Output:
264 Validation failed. Choice is not valid.
265
266 # Run the script with the argument 1
267 python3 03_validate.py -m 1
268
269 # Output:
270 Validation successful. Choice is valid.
271
272 # Run the script with the argument 2
273 python3 03_validate.py -m 2
274
275 # Output:
276 Validation successful. Choice is valid.
277
278 # Run the script with the argument 3
279 python3 03_validate.py -m 3
280
281 # Output:
282 Validation successful. Choice is valid.
283
284 # Run the script with the argument help
285 python3 03_validate.py -m help
286
287 # Output:
288 Validation failed. Choice is not valid.
289
290 # Run the script with the argument invalid
291 python3 03_validate.py -m invalid
292
293 # Output:
294 Validation failed. Choice is not valid.
295
296 # Run the script with the argument 1
297 python3 03_validate.py -m 1
298
299 # Output:
300 Validation successful. Choice is valid.
301
302 # Run the script with the argument 2
303 python3 03_validate.py -m 2
304
305 # Output:
306 Validation successful. Choice is valid.
307
308 # Run the script with the argument 3
309 python3 03_validate.py -m 3
310
311 # Output:
312 Validation successful. Choice is valid.
313
314 # Run the script with the argument help
315 python3 03_validate.py -m help
316
317 # Output:
318 Validation failed. Choice is not valid.
319
320 # Run the script with the argument invalid
321 python3 03_validate.py -m invalid
322
323 # Output:
324 Validation failed. Choice is not valid.
325
326 # Run the script with the argument 1
327 python3 03_validate.py -m 1
328
329 # Output:
330 Validation successful. Choice is valid.
331
332 # Run the script with the argument 2
333 python3 03_validate.py -m 2
334
335 # Output:
336 Validation successful. Choice is valid.
337
338 # Run the script with the argument 3
339 python3 03_validate.py -m 3
340
341 # Output:
342 Validation successful. Choice is valid.
343
344 # Run the script with the argument help
345 python3 03_validate.py -m help
346
347 # Output:
348 Validation failed. Choice is not valid.
349
350 # Run the script with the argument invalid
351 python3 03_validate.py -m invalid
352
353 # Output:
354 Validation failed. Choice is not valid.
355
356 # Run the script with the argument 1
357 python3 03_validate.py -m 1
358
359 # Output:
360 Validation successful. Choice is valid.
361
362 # Run the script with the argument 2
363 python3 03_validate.py -m 2
364
365 # Output:
366 Validation successful. Choice is valid.
367
368 # Run the script with the argument 3
369 python3 03_validate.py -m 3
370
371 # Output:
372 Validation successful. Choice is valid.
373
374 # Run the script with the argument help
375 python3 03_validate.py -m help
376
377 # Output:
378 Validation failed. Choice is not valid.
379
380 # Run the script with the argument invalid
381 python3 03_validate.py -m invalid
382
383 # Output:
384 Validation failed. Choice is not valid.
385
386 # Run the script with the argument 1
387 python3 03_validate.py -m 1
388
389 # Output:
390 Validation successful. Choice is valid.
391
392 # Run the script with the argument 2
393 python3 03_validate.py -m 2
394
395 # Output:
396 Validation successful. Choice is valid.
397
398 # Run the script with the argument 3
399 python3 03_validate.py -m 3
400
401 # Output:
402 Validation successful. Choice is valid.
403
404 # Run the script with the argument help
405 python3 03_validate.py -m help
406
407 # Output:
408 Validation failed. Choice is not valid.
409
410 # Run the script with the argument invalid
411 python3 03_validate.py -m invalid
412
413 # Output:
414 Validation failed. Choice is not valid.
415
416 # Run the script with the argument 1
417 python3 03_validate.py -m 1
418
419 # Output:
420 Validation successful. Choice is valid.
421
422 # Run the script with the argument 2
423 python3 03_validate.py -m 2
424
425 # Output:
426 Validation successful. Choice is valid.
427
428 # Run the script with the argument 3
429 python3 03_validate.py -m 3
430
431 # Output:
432 Validation successful. Choice is valid.
433
434 # Run the script with the argument help
435 python3 03_validate.py -m help
436
437 # Output:
438 Validation failed. Choice
```

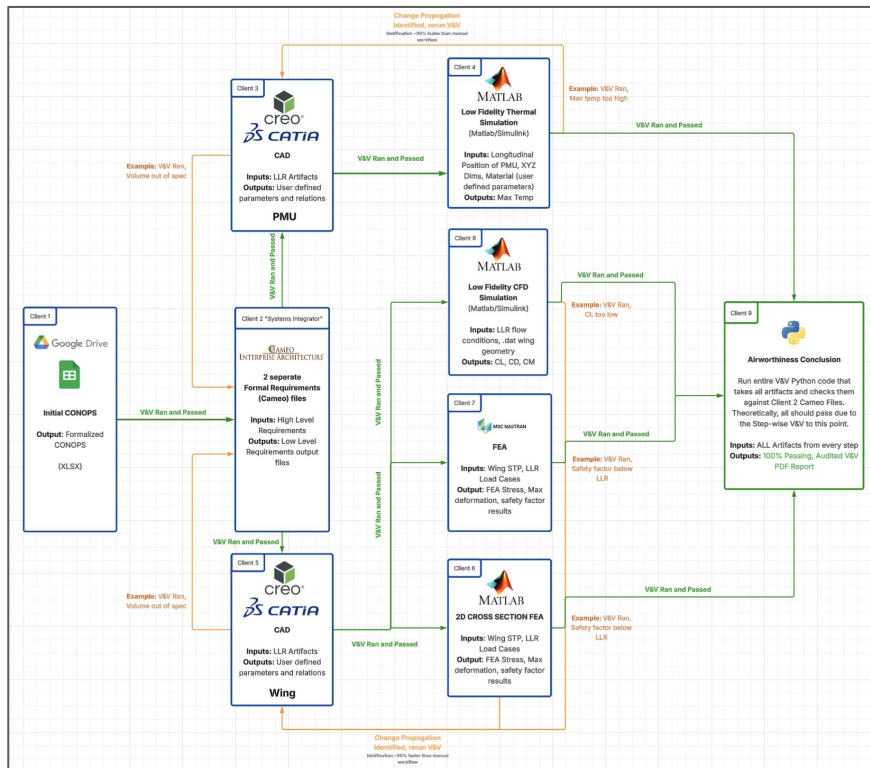
LLM Confidence Metrics

Outcome Alignment Rate: 100.0% Items both LLMs PASS: 3 Items both LLMs FAIL: 0

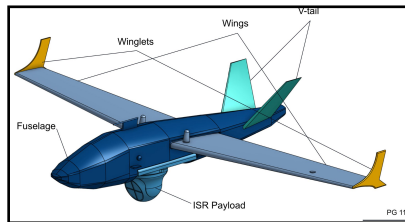
Validation Discrepancies - Manual Review Recommended

Item	ChatGPT	Claude	Reasons
PTC_MP_VAL_IMP_VOLUME	Not Found	PASS	ChatGPT: Not found by ChatGPT... Claude: Tire volume parameter found but no requirements specified (nan values)...

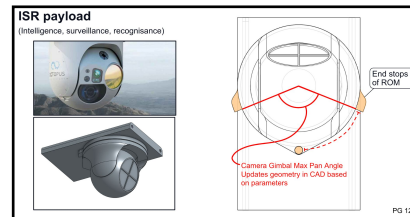
Federal Customer Success Project - "Industry One"



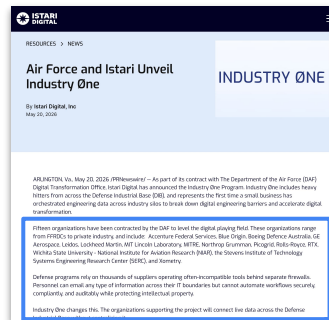
Project Workflow



UAV CAD Developed for Industry One



Payload CAD developed for Industry One



Fifteen organizations have been contracted by the DAF to level the digital playing field. These organizations range from FFRDCs to private industry, and include: Accenture Federal Services, Blue Origin, Boeing Defence Australia, GE Aerospace, Leidos, Lockheed Martin, MIT Lincoln Laboratory, MITRE, Northrop Grumman, Picogrid, Rolls-Royce, RTX, Wichita State University - National Institute for Aviation Research (NIAR), the Stevens Institute of Technology Systems Engineering Research Center (SERC), and Xometry.

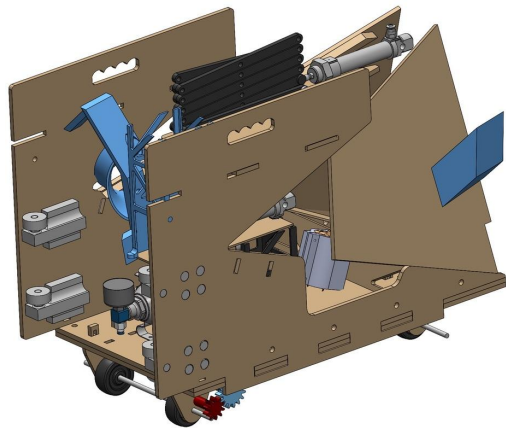
Defense programs rely on thousands of suppliers operating often-incompatible tools behind separate firewalls. Personnel can email any type of information across their IT boundaries but cannot automate workflows securely, compliantly, and auditably while protecting intellectual property.

Industry One changes this. The organizations supporting the project will connect live data across the Defense Industrial Base without centralizing it.

Department Wide Robotics Competition

Summary

- Designed automated robot "Light Fury" with continuous scissor lift (32" vertical extension, <0.5s deployment), drivetrain, and fish/eel sorting mechanisms using 3D-printed components and laser-cut MDF.
- Programmed fully autonomous operation in Arduino IDE (C++) using DC motors, pneumatic actuators, and solenoids for multi-stage task completion.
- Led team through sprint-based design evolution, creating specification sheets, House of Quality, and FMEA documentation, culminating in Final Design Review presentation.



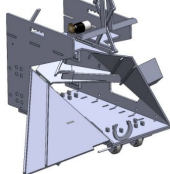
Design Evolution

Sprint 1



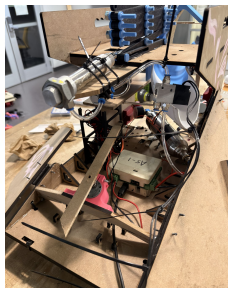
Foldable PVC Fish Routing
Hinge Supported Fish Sorter
MDF Laser Cut Drivetrain

Sprint 2



Laser Cut Fish & Eel Routing/Sorter
Dependent Systems

Sprint 3 + Final

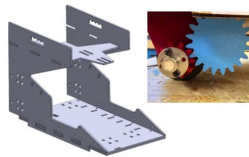


LIGHT FURY

Mio Achache, Loan Hoang, Abraham Peña, Kevin Son

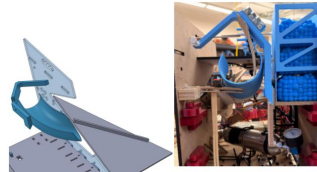
Design Implementation

Chassis + Drivetrain



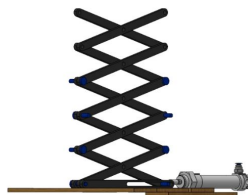
Interlocking Tabs for Assembly
3D Printed Gears (1.5 : 1)
Large DC Motor - Coupling Mounts
4s Deployment

Great Hall Interface



3D Printed Fish & Eel Sorter Arm
3D Printed Fish Routing Slide
60% Collection Zone Success
Pneumatic-Powered Sheep

Vertical Extension

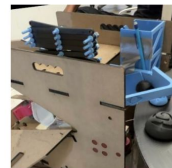


Pneumatic-Powered
32" of Vertical Extension
Deploys <0.5s

Horizontal Extension



Small DC Motor
Integrated Bonfire Movement



Fabrication

Staged approach to reduce risks.

Fabrication accomplished using laser cutting and 3D-printing: precise, easy to assemble, intricate cuts.

Modifications made with power tools and scrap pieces.

Organized planning for team collaboration.

Async Coding



My Journey So Far

